VTool-R1: VLMs Learn to Think with Images via Reinforcement Learning on Multimodal Tool Use

Paper by M. Wu, J. Yang, J. Jiang, M. Li, K. Yan, H. Yu, M. Zhang, C. Zhai, K. Nahrstedt
(UIUC & U. Michigan) arXiv:2505.19255

Presented by Shayan Salehi

Department of Computer Engineering
Sharif University of Technology

October 2026

Outline

1. Introduction & Problem Statement
2. Methodology
3. Experiments & Results
4. Conclusion & Future Work
5. References

Why Do VLMs Still "Think" Only in Text?

Background:

- RL fine-tuning (RFT) gave LLMs long CoT, self-correction and **learned tool use** (Search-R1).
- Recent VLM "R1" works (R1-V, Vision-R1, R1-VL, ...) copy this recipe.
- **But their reasoning stays text-driven:** the image is encoded once; every later step is text.

Why it matters

- A *six-fingered* hand: "how many fingers?" $\rightarrow$ the text prior "five" can beat the pixels.
- Inference-time visual CoT (Visual Sketchpad, ReFocus) helps, but only with GPT-4o-class models; small open models fail.

The Core Question

Can we **train** a small open VLM, with RL and only an outcome reward, to produce *intermediate visual steps* (edited images) and reason over them, i.e. to "think with images"?

The Missing Piece

No prior RFT framework puts image-editing tool calls *inside* the rollout, so the model never learns *when* or *how* to look again.

Three Roadblocks to Multimodal Chains of Thought

1. **Language shortcuts**
   Text-only CoT lets the decoder drift to a plausible-sounding answer that the image contradicts; nothing forces the model to re-inspect visual evidence.

2. **Prompting does not transfer to small models**
   Prompted tool use works for GPT-4o, but open Qwen2.5-VL 3B/7B fail to follow tool-use instructions: prompting them to use tools *lowers* accuracy (3B chart: 51.8 $\rightarrow$ 24.6).

3. **Training with tools in the loop is awkward**
   Generation must pause for external Python execution, the edited image must be fed back as a new input, and rewards on the tool calls themselves invite reward hacking.

Proposed Solution: VTool-R1

Contribution 1: RFT Framework with Visual Tools

- Python image-editing tools are executed *during* RL rollouts.
- Two-pass rollout: reason $\rightarrow$ edit image $\rightarrow$ reason over original $\oplus$ edited image.
- Built on GRPO; trained with an **outcome-only** 0/1 reward.

Contribution 2: First Learned "Thinking with Images"

- Claimed first work to make VLMs *learn* to interleave visual steps with text CoT.
- Model decides *whether* a tool helps; it is never rewarded for using one.
- Validated on chart and table QA (ReFocus splits), Qwen2.5-VL 3B/7B/32B.

Goal: turn "visual editing as chain of thought" from a GPT-4o-only prompting trick into a skill that small open VLMs acquire through RL.

Thinking with Images: What a Trained Rollout Looks Like

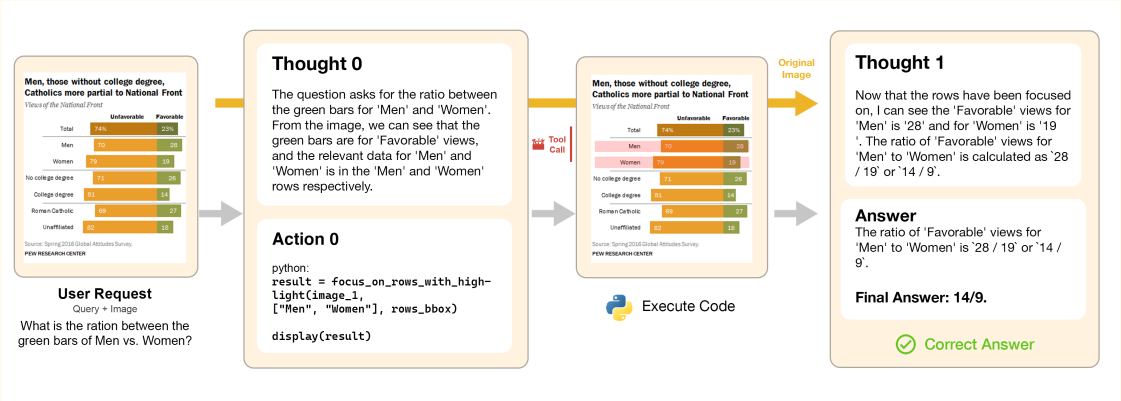
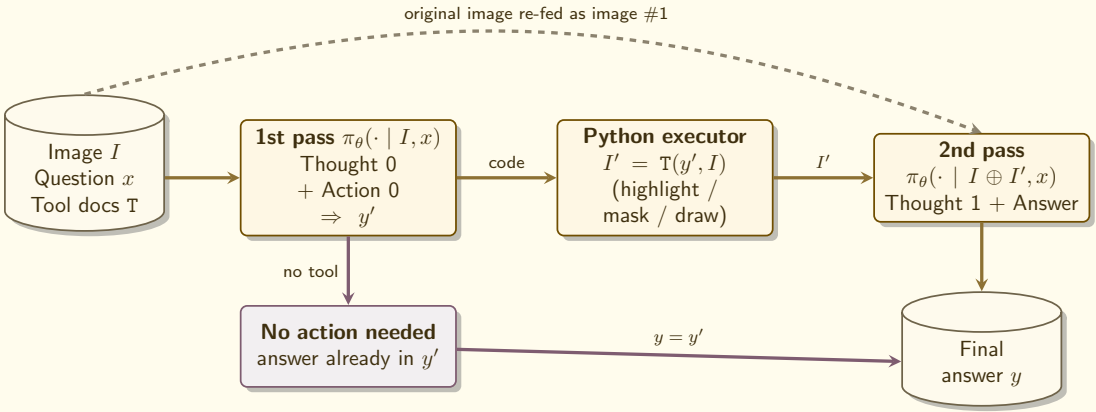


Figure: After RFT, the 3B model reasons in *Thought 0*, emits a tool call that highlights the "Men" and "Women" rows, receives the edited chart *alongside* the original, and reads the "Favorable" values (28 vs. 19) in *Thought 1*. The visual step narrows attention to exactly the evidence the question needs.

Inference & Rollout with a Visual Chain of Thought



$$y' \sim \pi_\theta(\cdot \mid I, x), \qquad I' = \mathtt{T}(y', I)$$

$$y \sim \pi_\theta(\cdot \mid I, x; \mathtt{T}) = \pi_\theta\big(\cdot \mid I \oplus \mathtt{T}(y', I),\, x\big)$$

- $\oplus$: original and edited image given as **two image inputs** (needs a multi-image VLM).
- At most **one** tool turn; several tool calls may be chained inside it.

Prompt Format and the Visual Editing Toolset T

Structured output format

```
THOUGHT 0:  rows ‘‘Men’’ and ‘‘Women’’ hold
the answer
ACTION 0:
  r = focus_on_rows_with_highlight(
      image_1, ["Men","Women"], rows_bbox)
  display(r)
OBSERVATION: <edited image = image #2>
THOUGHT 1:  Men = 28, Women = 19...
ANSWER: ... FINAL ANSWER: 14/9
TERMINATE
```

Prompt includes tool signatures, 9 requirements (e.g. “only one ACTION”), few-shot examples, and the **bounding boxes** of rows/columns (or chart bars).

Tools (Python + OpenCV, from ReFocus)

Tool	Applies to	Effect
Highlight	columns / rows	semi-transparent red overlay
Mask	columns / rows	white mask over irrelevant parts
Draw	columns / rows	solid red bounding box
Chart variants	bars (by x/y value)	highlight / mask bars

Intuition

Tools simulate **selective visual attention**: the model edits the image to isolate the evidence, then reads it again, as a human would with a highlighter.

Multi-Modal GRPO with Tool Use: Training Pipeline

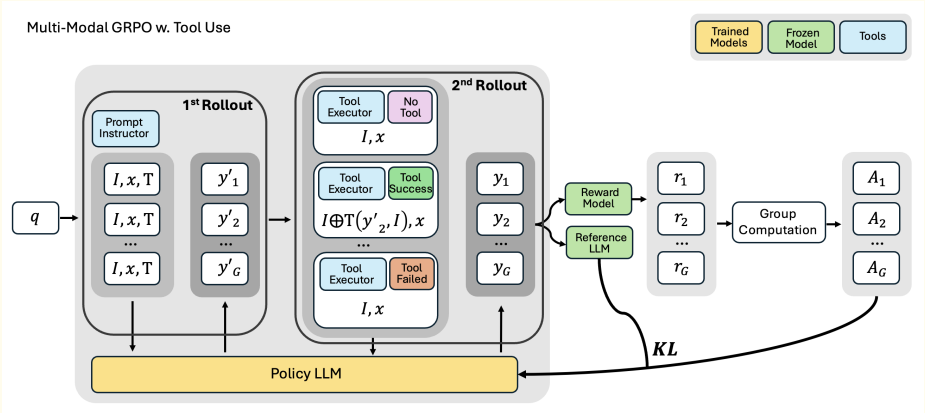


Figure: For each query q, G first-pass responses y'_i go through the tool executor: *no tool* (answer from I, x), *tool success* (2nd pass on $I \oplus \mathrm{T}(y'_i, I)$), or *tool failed* (fall back to I, x). Only the final y_i are scored, turned into group-relative advantages A_i, and used to update the policy under a KL penalty to the frozen reference.

Core Design Choices and Why

1. Optimize only the final response y

- The intermediate tool-call text y' is not directly optimized.
- *Why:* the goal is final answer quality; the model should decide on its own *whether* a tool helps. More end-to-end.

2. Edited image fed back as a 2nd input

- Unlike Search-R1, the tool output is not spliced into the token sequence.
- *Why:* the VLM must jointly attend to I and I'; requires multi-image capability.

3. Single tool turn

- One ACTION, possibly with several chained edits.
- *Why:* multi-turn editing needs stronger multi-image VLMs; left as future work.

4. GRPO, no format reward

- Critic-free group baseline: cheaper and more stable than PPO for 16k-token, multi-image rollouts.
- Structured template + few-shot examples already give reliable formatting.

Central insight

Tools are only *available* in the rollout and only correctness is rewarded, so tool use survives only where it actually helps.

Training Objective

RL objective with tools in the rollout (KL-regularized, Bradley–Terry reward r_ϕ):

$$\max_{\pi_\theta} \; \mathbb{E}_{[I,x]\sim\mathcal{D},\; y\sim\pi_\theta(\cdot|I,x;\mathtt{T})}\big[r_\phi(I,x,y)\big] \;-\; \beta\,\mathbb{D}_{\mathrm{KL}}\big[\pi_\theta(\cdot \mid I,x;\mathtt{T}) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid I,x;\mathtt{T})\big]$$

GRPO surrogate, with a group $\{y_i\}_{i=1}^{G} \sim \pi_{\mathsf{old}}(\cdot \mid I,x;\mathtt{T})$:

$$\mathcal{J}_{\mathsf{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\min\Big(\rho_{i,t}(\theta)\hat{A}_{i,t},\; \mathrm{clip}\big(\rho_{i,t}(\theta),1-\epsilon,1+\epsilon\big)\hat{A}_{i,t}\Big) - \beta\,\mathbb{D}_{\mathrm{KL}}\big[\pi_\theta \,\|\, \pi_{\mathrm{ref}}\big]\right]$$

$$\hat{A}_{i,t} = \frac{r_i - \mathrm{mean}(\mathbf{r})}{\mathrm{std}(\mathbf{r})}$$

- $\rho_{i,t}$: token-level probability ratio $\pi_\theta/\pi_{\mathsf{old}}$ (written $r_{i,t}$ in the paper).
- $\hat{A}_{i,t}$: same advantage for every token of y_i, no critic.
- Only tokens of the final y_i get gradient; I' comes from the environment.

Reward Design: Outcome Only, and Why Process Rewards Fail

Reward: $r_i = 1$ if a lightweight **LLM judge** says the predicted answer matches the ground truth, else 0 ("pseudo rule-based"; string match is too brittle for free-form chart/table answers).

Outcome reward (used)

0/1 on final-answer correctness only.

Tool use rises and falls with its usefulness; accuracy improves steadily.

Penalize failed tool calls

Model learns the safest policy:

tool usage collapses to zero.

Bonus for successful tool use

Model emits calls that merely *trigger* the success check;

hacking persists even with a stricter verifier.

Takeaway

Rewarding the *process* of tool use is easy to game. Rewarding only the *outcome* is the most robust signal found by the authors (evidence is reported qualitatively, without a table).

Experimental Setup

Data (exact ReFocus splits)

- **Table split:** VWTQ (750 QA, Wikipedia tables), VWTQ_syn (250, synthetic styles), VTabFact (250, entailment). 70/30 train/test, no validation set.
- **Chart split:** ChartQA; 14,344 train / 813 val; test = 444 horizontal + 382 vertical bar charts.

Models & baselines

- Qwen2.5-VL 3B / 7B / 32B: *Pure Run* (no tools) and *Tool Use* (prompted, no RFT; regenerate on failed call).
- R1-VL 2B/7B (general visual RL), DeepEyes 7B (concurrent), GPT-4o (upper bound).

Training (VeRL)

Optimizer	AdamW, lr 10^{-6}, wd 10^{-2}
Max sequence	16,384 tokens
Global batch	32 (table) / 256 (chart)
Group size G	5
KL coeff. β	10^{-2}
Decoding	temperature 1.0, bf16
Hardware	3B/7B: 8/16×H100; 32B: 8×H200

Metric: answer accuracy (%) judged against ground truth.

Main Results: Does RFT Teach Useful Tool Use?

Accuracy (%) of Qwen2.5-VL before and after VTool-R1

Method	3B		7B		32B	
	Chart	Table	Chart	Table	Chart	Table
Pure Run (no tools)	51.8	41.3	76.2	64.7	**88.0**	**86.2**
Tool Use (prompted)	24.6	24.3	53.4	41.1	85.0	76.0
VTool-R1 (RFT)	**64.0**	**57.9**	**80.7**	**71.7**	86.7	84.5

Key Takeaways:

- **+12.2 / +16.6** points over Pure Run at 3B (chart / table); **+4.5 / +7.0** at 7B.
- Prompting small models to use tools **costs 20–27 points**; RFT turns the same tools from a liability into a gain (+39.4 / +33.6 at 3B).
- **At 32B the gain disappears:** VTool-R1 recovers most of the tool-use drop but stays 1.3 / 1.7 points below Pure Run.

Comparison with Other RL-Tuned VLMs

Model	Size	Chart	Table
R1-VL (general visual RL)	2B	10.4	8.6
R1-VL (general visual RL)	7B	63.8	45.4
DeepEyes (RL + zoom tools)	7B	60.0	–
VTool-R1	3B	64.0	57.9
VTool-R1	**7B**	**80.7**	**71.7**
GPT-4o, Pure Run (upper bound)	–	82.9	75.7
GPT-4o, Tool Use (ReFocus)	–	80.5	77.0

- VTool-R1 3B already beats R1-VL 7B; the 7B model is within **2.2 / 4.0** points of GPT-4o.
- vs. concurrent DeepEyes: **80.7 vs. 60.0** on charts, attributed to tools and tasks matched to structured images.
- Caveat: R1-VL and DeepEyes were not trained on these splits, so part of the gap is in-domain training, not tool use alone.

Training Dynamics: Tool Use Is Not Monotonic

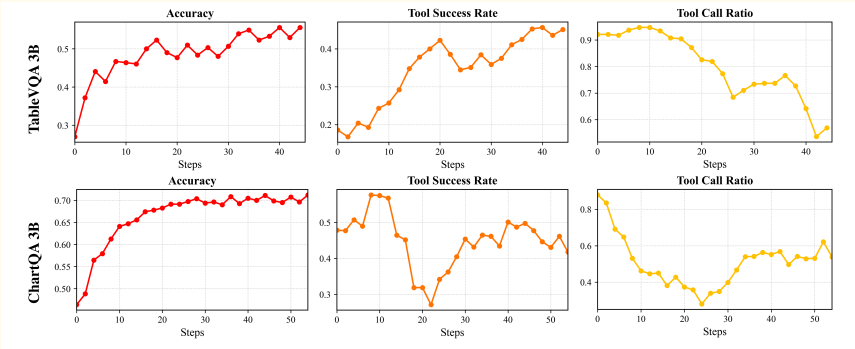
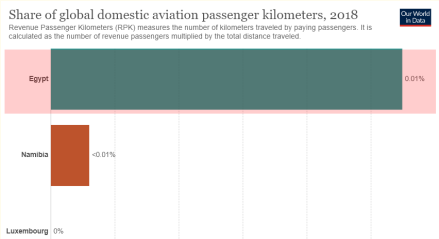


Figure: 3B training curves, TableVQA (top) and ChartQA (bottom). Accuracy rises (table ≈0.27→0.55, chart ≈0.46→0.71) and saturates in ∼50 steps, while the tool-call ratio *falls* (table 0.92→0.57; chart dips to 0.28, recovers to ∼0.55).

- Prompt-driven over-use early on; later the model is **selective** and skips tools when unnecessary.
- Tool success (proxy: code runs, returns an image) climbs on tables, fluctuates on charts; 32B keeps a higher call ratio.

Failure Analysis



Case (a). Q: "What is the value of the largest bar?" The tool correctly highlights Egypt and the 1st pass reads **0.01**, but the 2nd pass on the edited image says ≈0.001 and outputs "Egypt". GT: 0.01.

Four failure categories

(a) **Right visual step, wrong inference:** the 2nd rollout misreads or overrides a correct first-pass reading.

(b) **Flawed edit:** the drawn box occludes the numbers it was meant to emphasize.

(c) **Wrongly skips tools:** decides "no action needed" and answers incorrectly.

(d) **Invalid code:** hallucinated APIs (e.g. `get_children()`) fail, so no visual step is produced.

Observation

Since y' gets no gradient, a correct first-pass reading can be lost in the second pass; the visual step helps only if the 2nd pass actually uses it.

Current Limitations

Stated by the authors

- **Single tool turn only**: no iterative edit–reason loops.
- **Narrow scope**: selective attention on charts and tables with a fixed toolset.
- **No oracle for tool correctness**: "success" only means the code ran.

Our observations

- **Tools need given bounding boxes**: row/column/bar boxes are supplied in the prompt, which simplifies grounding.
- **No benefit at 32B**: gains shrink with scale; Pure Run wins at 32B.
- **Missing ablation**: no RFT run *without* tools, so the gain from RL alone vs. RL + tools is not isolated.
- **Reward-design evidence is qualitative**; the LLM judge is itself a possible noise source.

Conclusion & Takeaways

1. **Framework:** VTool-R1 puts Python image-editing tools inside GRPO rollouts, so a VLM reasons over the original image $\oplus$ its own edited image.

2. **Outcome-only reward is enough:** the model learns *when* to use tools without process supervision, while process rewards get hacked.

3. **Results:** 3B/7B Qwen2.5-VL gain up to **+16.6** points over direct answering; 7B approaches GPT-4o on chart and table QA.

4. **Outlook:** multi-turn editing, richer (generative) tools, and broader tasks beyond structured images.

Thank You!

Questions?

References I

Wu, M., Yang, J., Jiang, J., et al. (2025). VTool-R1: VLMs Learn to Think with Images via Reinforcement Learning on Multimodal Tool Use. *arXiv:2505.19255*.

Fu, X., Liu, M., Yang, Z., et al. (2025). ReFocus: Visual Editing as a Chain of Thought for Structured Image Understanding. *ICML*.

Hu, Y., Shi, W., Fu, X., et al. (2024). Visual Sketchpad: Sketching as a Visual Chain of Thought for Multimodal Language Models. *NeurIPS*.

DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*.

Shao, Z., Wang, P., Zhu, Q., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*.

Schulman, J., Wolski, F., Dhariwal, P., et al. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.

Jin, B., Zeng, H., Yue, Z., et al. (2025). Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. *arXiv:2503.09516*.

Bai, S., Chen, K., Liu, X., et al. (2025). Qwen2.5-VL Technical Report. *arXiv:2502.13923*.

References II

- Zhang, J., Huang, J., Yao, H., et al. (2025). R1-VL: Learning to Reason with Multimodal Large Language Models via Step-wise Group Relative Policy Optimization. *arXiv:2503.12937*.
- Zheng, Z., Yang, M., Hong, J., et al. (2025). DeepEyes: Incentivizing "Thinking with Images" via Reinforcement Learning. *arXiv:2505.14362*.
- Sheng, G., Zhang, C., Ye, Z., et al. (2024). HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv:2409.19256*.
- Masry, A., Long, D. X., Tan, J. Q., et al. (2022). ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning. *Findings of ACL*.
- Kim, Y., Yim, M., & Song, K. Y. (2024). TableVQA-Bench: A Visual Question Answering Benchmark on Multiple Table Domains. *arXiv:2404.19205*.
- Pasupat, P., & Liang, P. (2015). Compositional Semantic Parsing on Semi-Structured Tables. *ACL*.
- Chen, W., Wang, H., Chen, J., et al. (2020). TabFact: A Large-scale Dataset for Table-based Fact Verification. *ICLR*.